Available at
www.ElsevierComputerScience.com
POWERED BY SCIENCE DIRECT®

Journal of Algorithms 50 (2004) 106–117

Journal of Algorithms

www.elsevier.com/locate/jalgor

An efficient parameterized algorithm for m-set packing ☆

Weijia Jia,[a,*] Chuanlin Zhang,[a,b,1] and Jianer Chen [c,2]

[a] *Department of Computer Engineering and IT, City University of Hong Kong, Hong Kong of China*
[b] *Department of Mathematics, Jinan University, Guangzhou 510632, PR China*
[c] *Department of Computer Science, Texas A&M University, College Station, TX 77843-3112, USA*

Received 24 April 2003

Abstract

We present an efficient parameterized algorithm solving the Set Packing problem, in which we assume that the size of the sets is bounded by m. In particular, if the size m of the sets is bounded by a constant, then our algorithm is fixed-parameter tractable. For example, if the size of the sets is bounded by 3, then our algorithm runs in time $O((5.7k)^k n)$.

Keywords: Parameterized computation; Set packing; Exact algorithm; NP-hard problem

1. Introduction

Parameterized computation [3,4,7] has drawn much attention recently in dealing with problems that are NP-hard or worse—as is so frequently the case in the natural world of computing. The study was motivated by the observation that many computational problems are associated with an important parameter k that is bounded in a small or moderate range in practical applications. Therefore, taking the advantage of the small size of the parameter

☆ Supported in part by UGC of Hong Kong under Grants (No. CityU 1039/02E, CityU 1055/01E), and CityU HK Grant (No. 7001355).
* Corresponding author.
E-mail addresses: itjia@cityu.edu.hk (W. Jia), chen@cs.tamu.edu (J. Chen).
1 Supported in part by Main Subject Foundation of the State Council's Office of Overseas Chinese Affairs and the Doctoral Foundation under Grants 93A109 and 98-D1029. Part of the work was done while this author was visiting City University of Hong Kong.
2 Supported in part by the USA National Science Foundation under the grant CCR-0000206.

0196-6774/$ – see front matter
doi:10.1016/j.jalgor.2003.07.001

may significantly speedup the computation, thus solving computational problems that are intractable in their general form. A well-known example of this kind of problems is the vertex cover problem (deciding whether a given graph has a vertex cover of size k). The vertex cover problem is NP-hard in its general form [8], but in an important application in biochemistry, we are only interested in knowing whether a given graph, which can be very large, has a vertex cover of at most 60 vertices [7]. A number of algorithms have been proposed for solving the vertex cover problem in terms of this particular application (see, e.g., [1,5,6]). In particular, the vertex cover problem now can be solved in time $O(1.285^k + kn)$ [6], which is pretty much practical for applications where k is not larger than 100.

In this paper, we study the parameterized complexity for another well-known NP-hard problem, the set packing problem. The set packing problem is NP-complete in its general form even if the size of the sets in the collection is bounded by 3 (see [8,11]). An instance of the parameterized set packing problem (PSP) is a pair (C, k), asking whether the given collection C of sets contains k mutually disjoint sets. Here we assume that k is much smaller than the total number n of sets in the collection C. A straightforward algorithm with time complexity $O(n^k N)$ can be derived by examining all sub-collections of k sets in C, where N is the total number of elements in the universal set μ. The algorithm is not practically feasible even for small values of k (e.g., $k = 10$). On the other hand, the PSP problem is $W[1]$-complete, which means that we cannot expect an algorithm much better than the above straightforward algorithm [7].

Therefore, to develop more efficient algorithms for the PSP problem, we need to enforce further constraints. One natural constraint is to restrict the size of the sets in the collection C. Note that this constraint has been also used in the study of the NP-completeness of the original set packing problem [8,11]. Thus, from now on we assume that each instance of the PSP problem is a triple (C, m, k), where C is a finite collection of sets, each contains exactly m elements, and asking whether the collection C contains k mutually disjoint sets. We will call this version of the PSP problem the *parameterized* m-set packing problem (m-PSP).

In this paper, we apply a new technique to solve the m-PSP problem. We start with a carefully designed nondeterministic algorithm for the problem, then use a smart "de-nondetermination" technique to convert the nondeterministic algorithm into an efficient deterministic algorithm for the m-PSP problem. The nondeterministic algorithm provides us with very simple and clear logic and guideline for our design and correctness verification, while the de-nondetermination technique effectively converts the simple logic nondeterministic algorithm into an efficient deterministic algorithm for the problem. The running time of our deterministic algorithm is bounded by $O(g(k, m)n)$, where $g(k, m)$ is a function only depending on the parameter k and the set size m but independent of the number n of sets in the given collection C. Thus, in case the size m of the sets is bounded by a constant, our algorithm is fixed-parameter tractable [7]. In particular, in case the size m of the sets in the collection C is bounded by 3, our algorithm runs in time $O((5.7k)^k n)$.

We give the notations and terminologies we will use in this paper. Let μ be the universal set in our discussion, i.e., all elements in the sets in a collection C for the m-PSP problem are elements in μ. We will use small English letters such as a to denote elements in the universal set μ, small Greek letters such as σ to denote the sets of elements in μ, and

capitalized English letters such as C to denote the collections of sets. Let C be a collection of sets. A *set packing* in C is a sub-collection P of C such that any two sets in P are disjoint. A *partial set* σ^* is a set σ in C with zero or more elements in σ replaced by the symbol "$*$". A set in C without "$*$" is also called a *regular set*. Note that by the definition a regular set is also a partial set. The set consisting of the non-"$*$" elements in a partial set σ^* is denoted as $\text{reg}(\sigma^*)$. A partial set σ^* is *consistent with* a regular set σ if $\text{reg}(\sigma^*) \subseteq \sigma$. A collection P of partial sets is called a *partial set packing* if P is obtained from a set packing M in C by replacing in the sets in M certain elements with "$*$". In this case, we say that the partial set packing P is *consistent with* the set packing M. A set packing M in the collection C is *k-maximal* if either (1) $|M| = k$; or (2) $|M| < k$ and for every set $\sigma \in C - M$, there is at least one set $\sigma' \in M$ such that $\sigma \cap \sigma' \neq \emptyset$.

2. A nondeterministic algorithm

Let (C, m, k) be an instance of the m-PSP problem, where C is a collection of n sets, each contains exact m elements from the universal set μ. Our algorithm starts with a procedure that constructs a k-maximal set packing M_0 in the collection C. The procedure is given in Fig. 1.

The following lemma can be easily verified.

Lemma 2.1. *Algorithm* I *runs in time $O(mn)$ and constructs a k-maximal set packing M_0 in C.*

The following lemma follows directly from the definition of k-maximal set packings, and will play an important role in our discussion.

Lemma 2.2. *Let M_0 be a k-maximal set packing and let M_k be a set packing of k sets. If $|M_0| < k$, then for every set σ in M_k, there is a set σ' in M_0 such that $\sigma \cap \sigma' \neq \emptyset$.*

By Lemma 2.1, Algorithm I returns a k-maximal set packing M_0 in C. There are two cases in which we can directly decide whether (C, m, k) is a "yes" or "no"-instance of the m-PSP problem:

Algorithm I. k-maximal.
Input: an instance (C, m, k) of the m-PSP problem.
Output: a k-maximal set packing M_0 in C
1. $M_0 = \emptyset$;
2. *for* each set σ in C *do*
    if no element in σ is marked "used"
    then {$M_0 = M_0 \cup \{\sigma\}$; mark all elements in σ as "used";
        if ($|M_0| = k$) *then* return(M_0); STOP.}
3. return(M_0); STOP.

Fig. 1. Constructing a k-maximal set packing.

Algorithm II. Greedy-replacing.
Input: a partial set packing P_k.
Output: a set packing M and a partial set packing Q_k
1. $Q_k = P_k$;
2. *for* each set σ in C *do*
   if there is a unique partial set σ^* in Q_k such that $\text{reg}(\sigma^*) \subseteq \sigma$ and σ does not intersect any other set except σ^* in Q_k
   then $Q_k = (Q_k - \{\sigma^*\}) \cup \{\sigma\}$;
3. let M be the collection of the regular sets in Q_k;
4. return M and Q_k.

Fig. 2. Converting a partial set packing to a regular set packing.

(1) if $|M_0| = k$, then certainly (C, m, k) is a "yes"-instance of the m-PSP problem;
(2) if $|M_0| < k/m$, then we can directly conclude that (C, m, k) is a "no"-instance of the m-PSP problem.

This is verified by the following lemma.

Lemma 2.3. *Let M_0 be a k-maximal set packing in the collection C, and $|M_0| < k/m$. Then there is no set packing of k sets in C.*

Proof. Suppose M_k is a set packing of k sets in C. By Lemma 2.2, for each set σ in M_k, there is a set σ' in M_0 such that $\sigma \cap \sigma' \neq \emptyset$. Since both M_k and M_0 are set packings (i.e., the sets in each of them are all disjoint), and M_k has k sets, there are at least k different elements in the sets in M_0. Now since each set contains exactly m elements, we derive that M_0 has at least k/m sets, contradicting the assumption on M_0. This proves the lemma. □

Therefore, we only need to consider the situation $k/m \leqslant |M_0| < k$, where M_0 is the k-maximal set packing returned by Algorithm I. Suppose the collection C has a set packing $M_k = \{\sigma_1, \sigma_2, \ldots, \sigma_k\}$ of k sets. Then by Lemma 2.2, each set σ_i in M_k contains an element a_i that is also contained in a set in M_0. Thus, we can use a nondeterministic algorithm to (correctly) guess the k elements $a_1, a_2, \ldots, a_k$ in the sets in M_0. This gives us the following collection:

$$P_k = \{\sigma_1^*, \sigma_2^*, \ldots, \sigma_k^*\}$$

where σ_i^* is a partial set containing the element a_i and $m-1$ "$*$" symbols. Note that if the collection C has a set packing M_k of k sets and if our nondeterministic algorithm guesses the symbols $a_1, a_2, \ldots, a_k$ correctly, then P_k is a partial set packing consistent with the set packing M_k of k sets.

Inductively, suppose we have a partial set packing $P_k = \{\sigma_1^*, \sigma_2^*, \ldots, \sigma_k^*\}$. We try to replace each partial set in P_k by a regular set in the collection C, using Algorithm II given in Fig. 2.

Again there are two possible cases for the outcome of Algorithm II. If $|M| = k$, then we have found a set packing of k sets, so we are done.

Now suppose $|M| = h < k$. Without loss of generality, suppose $P_k = \{\sigma_1^*, \sigma_2^*, \ldots, \sigma_k^*\}$ and $M = \{\sigma_1', \sigma_2', \ldots, \sigma_h'\}$, where the partial set σ_i^* in P_k was replaced by the regular set

Algorithm III. N-set-packing.
Input: an instance (C, m, k) of the m-PSP problem.
Output: a set packing of k sets or report no such a packing exists
1. call Algorithm I to construct a k-maximal set packing M_0;
2. *if* $|M_0| = k$ *then* return (M_0); STOP;
3. *if* $|M_0| < k/m$ *then* return("no such a packing"); STOP;
4. guess k elements $a_1, a_2, \ldots, a_k$ in the sets in M_0;
5. let $P_k = \{\sigma_1^*, \sigma_2^*, \ldots, \sigma_k^*\}$, where the partial set σ_i^* contains the element a_i and $m-1$ "$*$" symbols;
6. *while* any set in P_k still contains "$*$" symbols *do*
6.1. call Algorithm II on P_k to construct the set packing M and the partial set packing Q_k;
6.2. *if* $|M| = k$ *then* return(M); STOP;
6.3. *if* M contains no new element *then* return("no such a packing"); STOP;
6.4. pick a partial set σ^* in $Q_k - M$;
6.5. guess a new element a from M;
6.6. *if* replacing a "$*$" in σ^* by a gives a partial set consistent with at least one set in C
6.7. *then* in the collection P_k, replace a "$*$" symbol in the set σ^* by the element a;
6.8. *else* return("no such a packing"); STOP.

Fig. 3. The nondeterministic algorithm for set packing.

σ_i' by Algorithm II in step 2, for $1 \leqslant i \leqslant h$ (therefore, $Q_k - M = \{\sigma_{h+1}^*, \ldots, \sigma_k^*\}$). By our assumption, P_k is a partial set packing consistent with a set packing $M_k = \{\sigma_1, \sigma_2, \ldots, \sigma_k\}$ of k sets in C, such that $\text{reg}(\sigma_i^*) \subseteq \sigma_i$ for all $1 \leqslant i \leqslant k$. Now consider the set σ_{h+1} in M_k. The set σ_{h+1} does not intersect any partial set σ_i^* in P_k for $i \neq h+1$ since all sets σ_i, $1 \leqslant i \leqslant k$, are disjoint. Thus, $\text{reg}(\sigma_i^*) \not\subseteq \sigma_{h+1}$ for all $i \neq h+1$. In particular, no partial set σ_i^*, $1 \leqslant i \leqslant h$, was replaced by σ_{h+1} in Algorithm II. Now by our assumption, σ_{h+1} did not replace σ_{h+1}^* in Algorithm II, either. Thus, when σ_{h+1} was considered by Algorithm II, one σ_i' of the regular sets σ_1', σ_2', $\ldots$, σ_h' in M must already exist in Q_k and σ_{h+1} intersects σ_i'. Since σ_{h+1} does not intersect σ_i^*, σ_{h+1} must contain an element in $\sigma_i' - \text{reg}(\sigma_i^*)$ (recall that σ_i^* was replaced by σ_i' in Algorithm II).

Therefore, if we call the elements in the set $W = \bigcup_{i=1}^{h}[\sigma_i' - \text{reg}(\sigma_i^*)]$ "new elements" (W is the set of elements in M that are not in P_k), then one of the new elements must be contained in the set σ_{h+1}. Note that no new element can be in the partial set σ_{h+1}^* since otherwise the set containing the new element would have not been used by Algorithm II to replace a partial set in Q_k. Thus, if we correctly guess this new element a in W, then we can replace a "$*$" symbol in σ_{h+1}^* by the element a (note $\sigma_{h+1}^* \in Q_k - M$). This will eliminate one "$*$" symbol in the original partial set packing P_k and make P_k "one symbol closer" to the set packing M_k.

Thus, each execution of the process of running Algorithm II on the partial set packing P_k then correctly guessing a new element in M to replace a "$*$" symbol in P_k will either end up with a set packing M of k sets, or eliminate one "$*$" symbol in a set σ_{h+1} in P_k ($\sigma_{h+1} \in Q_k - M$). Therefore, after at most $k(m-1)$ executions of the process, we should eliminate all "$*$" symbols in P_k and obtain a set packing of k sets.

We summarize the above discussion in Algorithm III given in Fig. 3.

Theorem 2.4. *The nondeterministic Algorithm* III *solves the m-PSP problem correctly.*

Proof. According to the above discussion, if the collection C has a set packing M_k of k sets, then the computation path of Algorithm III that makes all correct guesses will eventually construct a set packing of k sets, and stop with a "yes" conclusion. By the definition of nondeterministic computation, this means that if the input (C, m, k) is a "yes"-instance of the m-PSP problem, then Algorithm III accepts (C, m, k) with a set packing of k sets constructed.

Now suppose that the collection C does not have a set packing of k sets. Note that a computation path of Algorithm III would not conclude with a "yes" unless it actually constructs a set packing of k sets in C. Thus, all computation paths will stop with a "no" answer. In particular, a computation path will either stop at step 3 when the k-maximal set packing M_0 has less than k/m sets, or stop at step 6.3 or step 6.8 when no "$*$" symbol in the partial set packing P_k can be further converted. Note that even in case the set packing M_k of k sets exists, a computation path with incorrect guesses may stop with a "no" answer. The point is, on a "yes"-instance, at least one computation path of Algorithm III ends up with a "yes" answer, while on a "no"-instance, all computation paths of Algorithm III will end up with a "no" answer. □

3. De-nondeterminating the algorithm

Roughly speaking, we eliminate the nondeterminism in Algorithm III by enumerating all possibilities. In fact, from the current understanding in theoretical computer science, essentially there has no better ways than exhaustive enumeration. However, for our algorithm, more effective enumeration and more thorough analysis can lead to a more efficient deterministic algorithm.

3.1. De-nondeterminating step 4 in Algorithm III

Suppose that the k-maximal set packing M_0 contains $h < k$ sets, each contains exactly m elements. A straightforward way of enumerating all selections of k elements $a_1, a_2, \ldots, a_k$ in the sets in M_0 will take time proportional to

$$\binom{mh}{k} \leqslant \binom{mk}{k} = O\big((mk)^k\big).$$

We describe below a more effective method of enumeration.

Let P be a set packing in C and let β be a set of elements in μ. We say that β is a *spanning set* for P if each element in β is contained in a set in P and each set in P contains at least one element in β. In particular, if the number of elements in β is equal to the number of sets in P, then β is a spanning set for P if and only if each set in P contains exactly one element in β.

Lemma 3.1. *Let M_0 be a k-maximal set packing in the collection C and $|M_0| = h < k$. If C has a set packing of k sets, then there is a set $\beta = \{a_1, a_2, \ldots, a_k\}$ of k elements such that β is a spanning set for M_0 and also a spanning set for a set packing of k sets in C.*

Proof. Let $M_{\max}$ be a maximum set packing in the collection C. Construct a bipartite graph $G = (V, E)$ with vertex bipartition $V = V_0 \cup V_{\max}$, where each vertex in V_0 is a set in M_0 and each vertex in $V_{\max}$ is a set in $M_{\max}$. There is an edge from a vertex in V_0 to a vertex in $V_{\max}$ if the intersection of the two corresponding sets in M_0 and $M_{\max}$ is not empty. We first show that for any vertex subset V_0' in V_0, the vertex subset $N(V_0')$ contains at least $|V_0'|$ vertices, where $N(V_0')$ is the set of vertices in $V_{\max}$ that are adjacent to at least one vertex in V_0'. Suppose the contrary that there is a vertex subset V_0' in V_0 such that $|V_0'| > |N(V_0')|$. Then the vertex set $W = V_0' \cup [V_{\max} - N(V_0')]$ would contain more than $|V_{\max}| = |M_{\max}|$ vertices. Since no set in V_0' intersects with any set in $V_{\max} - N(V_0')$, and both M_0 and $M_{\max}$ are set packings, we derive that W corresponds to a set packing of more than $|M_{\max}|$ sets in the collection C. This contradicts our assumption that $M_{\max}$ is a maximum set packing in C.

Thus, in the graph G, we have $|V_0'| \leqslant |N(V_0')|$ for all vertex subsets V_0' in V_0. By Hall's theorem [2], there is a matching in the graph G with $|V_0| = h$ edges: $[\sigma_1, \sigma_1']$, $[\sigma_2, \sigma_2']$, …, $[\sigma_h, \sigma_h']$, where $\sigma_i \in V_0$ and $\sigma_i' \in V_{\max}$, and suppose $a_i \in \sigma_i \cap \sigma_i'$, for all $i = 1, \ldots, h$. Now pick any $k - h$ sets σ_{h+1}', …, σ_k' in $M_{\max} - \{\sigma_1', \ldots, \sigma_h'\}$. Then $M_k = \{\sigma_1', \ldots, \sigma_h', \sigma_{h+1}', \ldots, \sigma_k'\}$ is a set packing of k sets in C. From each set σ_i', $i = h + 1, \ldots, k$, pick an element a_i such that a_i is contained in a set in M_0 (such an element a_i must exist since M_0 is k-maximal). Now it is easy to verify that the element set $\{a_1, \ldots, a_h, a_{h+1}, \ldots, a_k\}$ is a spanning set for both M_0 and M_k. □

By Lemma 3.1, instead of enumerating all selections of k elements in the sets in M_0, we can simply enumerate only the spanning sets of k elements for M_0.

Lemma 3.2. *The total number of spanning sets of k elements for the k-maximal set packing M_0 is bounded by b_m^k, where b_m is the minimum positive root of the equation $x^m = \sum_{i=1}^{m} \binom{m}{i} x^{m-i}$.*

Proof. Suppose $|M_0| = h$, and let $D(h, k)$ denote the number of spanning sets of k elements for M_0. We then have the following recurrence relation:

$$D(h,k) = \sum_{i=1}^{m} \binom{m}{i} D(h-1, k-i), \tag{1}$$

$$D(h,k) = \begin{cases} 0 & \text{if } h > k \text{ or } mh < k \text{ or } k \leqslant 0, \\ 1 & \text{if } mh = k. \end{cases} \tag{2}$$

Recurrence relation (1) can be explained as follows. Fix a set σ in M_0, and suppose we pick i elements from σ for the spanning set. For each selection of i elements in σ, we have $D(h-1, k-i)$ ways to select the remaining $k - i$ elements that make a spanning set for the rest $h - 1$ sets in M_0. The recurrence relation follows since there are $\binom{m}{i}$ ways to select i elements in the set σ.

Now we prove by induction on k that the solution of the recurrence relation (1)–(2) is bounded as $D(h, k) \leqslant b_m^k$, where b_m is the minimum positive root of the equation

$$x^m = \sum_{i=1}^{m} \binom{m}{i} x^{m-i}. \tag{3}$$

It is easy to see $D(h,k) \leqslant b_m^k$ for the boundary conditions $h > k$ or $mh < k$ or $k \leqslant 0$. Dividing both sides in Eq. (3) by x^{m-k}, we find that b_m is also a root of the equation

$$x^k = \sum_{i=1}^{m} \binom{m}{i} x^{k-i}.$$

Now using the inductive hypothesis, we have

$$D(h,k) = \sum_{i=1}^{m} \binom{m}{i} D(h-1,k-i) \leqslant \sum_{i=1}^{m} \binom{m}{i} b_m^{k-i} = b_m^k.$$

This completes the proof. □

The value b_m only depends on m, thus is a constant when m is a constant. For each fixed constant m, the root b_m of Eq. (3) can be computed using numerical method [12]. For example, $b_3 \approx 3.8473$ and $b_4 \approx 5.2853$.

As explained above, the $D(h,k)$ spanning sets of k elements for the k-maximal set packing M_0 can be enumerated systematically. We conclude that the guesses in step 4 of Algorithm III can be replaced by at most b_m^k deterministic enumerations.

3.2. De-nondeterminating step 6.5 in Algorithm III

We first show that some of the "$*$" symbols in the partial set packing P_k may be converted into elements in μ deterministically.

Lemma 3.3. *Suppose the collection C has a set packing of h sets, and let P_h be a partial set packing of h partial sets in which each partial set contains at most one "$*$". Then a set packing M_h consistent with P_h can be constructed from P_h deterministically in time $O(h^3 + mn)$.*

Proof. Without loss of generality, we assume each partial set in P_h contains exactly one "$*$" (otherwise, we remove the partial sets that contain no "$*$" and work on a smaller h). We construct a bipartite graph $G = (V, E)$ with vertex bipartition $V = V_L \cup V_R$. Each vertex in V_L is a partial set in P_h, and each vertex in V_R is an element a in μ such that (1) a is not contained in any partial set in P_h; and (2) there is a partial set σ in P_h such that replacing the "$*$" symbol in σ by a results in a regular set in C. There is an edge from a vertex σ in V_L to a vertex a in V_R if replacing the "$*$" symbol in σ by a results in a regular set in C.

We construct a maximum matching in the bipartite graph G. Since P_h is a partial set packing of h partial sets, P_h is consistent with a set packing M_h of h sets. Because of the existence of the set packing M_h, a maximum matching in G contains exactly $|V_L| = h$ edges. If a vertex σ in V_L has degree at least h, then we can remove the vertex σ from the graph and first construct a maximum matching in the remaining graph. This is because for any maximum matching in the remaining graph, we can easily add another edge incident on σ to make a maximum matching in G. Thus, with a preprocessing of time $O(mn)$, we can assume, without loss of generality, that each vertex in V_L has degree bounded by $h-1$. In consequence, the graph G has at most $h(h-1)$ edges and at most

Algorithm II'. Greedy-replacing.
Input: a partial set packing P_k.
Output: a set packing M and a partial set packing Q_k
1. let Q_1 be the collection of partial sets in P_k that contain at most one "$*$";
2. let C' be the collection of sets in C that do not intersect any set in $P_k - Q_1$;
3. construct a set packing M_1 of $|Q_1|$ sets in C' such that M_1 and Q_1 are consistent;
4. *if* there is no such a set packing M_1 *then* return("fail"); STOP.
5. $Q_k = M_1 \cup (P_k - Q_1)$;
6. *for* each set σ in C *do*
   if there is a unique partial set σ^* in Q_k such that $\text{reg}(\sigma^*) \subseteq \sigma$
   and σ does not intersect any other set except σ^* in Q_k
   then $Q_k = (Q_k - \{\sigma^*\}) \cup \{\sigma\}$;
7. let M be the collection of the regular sets in Q_k;
8. return M and Q_k.

Fig. 4. Modified conversion of a partial set packing to a regular set packing.

$h(h-1) + h = h^2$ vertices. By the maximum matching algorithm in [10], a maximum matching in the graph G, which has exactly h edges, can be constructed in time $O(h^3)$.

From the maximum matching in G, a set packing of h elements can be easily constructed: for each edge $[\sigma, a]$ in the maximum matching, where $\sigma \in V_L$ and $a \in V_R$, we simply replace the "$*$" symbol in σ by the element a to make a regular set in C. It is easy to verify that this gives a set packing of h sets in the collection C. □

In particular, from Lemma 3.3, if we have a partial set packing of k partial sets in which each partial set contains at most one "$*$", then we can construct a set packing of k sets in time $O(k^3 + mn)$. Therefore, if a partial set in the partial set packing P_k contains only one "$*$", then we do not have to convert it until every partial set in P_k contains only one "$*$".

This suggests a modification of Algorithm II, which is given in Fig. 4.

Lemma 3.4. *The running time of Algorithm* II' *is* $O(k^3 + kmn)$.

According to Lemma 3.3, if the set packing M_1 in step 3 of Algorithm II' does not exist, then the collection C has no set packing of $h \leqslant k$ sets. Otherwise, the set packing M_1 is constructed. At step 5 of Algorithm II' for the collection Q_k, the partial sets in Q_1 are all replaced by the regular sets in M_1 and each partial set in $P_k - Q_1$ contains at least two "$*$"s. In particular, at the end of Algorithm II', no partial set with a single "$*$" in P_k is left in $Q_k - M$. Therefore, if we replace Algorithm II by Algorithm II' in Algorithm III, then when Algorithm III picks a partial set σ^* in $Q_k - M$ in step 6.4, the partial set σ^* contains at least two "$*$"s. This guarantees that in step 6.7 of Algorithm III, a "$*$" symbol is converted in a partial set containing at least two "$*$"s. Of course, eventually, every partial set in P_k contains only one "$*$", then the set packing M_1 in step 3 of Algorithm II' contains k sets and $M = M_1$, so step 6.2 of Algorithm III will stop with a "yes" conclusion and a set packing M of k sets.

Now we are ready to present our deterministic algorithm, which is given in Fig. 5 (compare this algorithm with the nondeterministic algorithm Algorithm III in Fig. 3). Note that the statements in Algorithm IV have been numbered in a way that makes the comparisons of the algorithms easy.

Algorithm IV. D-set-packing.
Input: an instance (C, m, k) of the m-PSP problem.
Output: a set packing of k sets or report no such a packing exists
1. call Algorithm I to construct a k-maximal set packing M_0;
2. *if* $|M_0| = k$ *then* return (M_0); STOP;
3. *if* $|M_0| < k/m$ *then* return("no such a packing"); STOP;
4. *for* each spanning set $\{a_1, a_2, \ldots, a_k\}$ for M_0 *do*
5. let $P_k = \{\sigma_1^*, \sigma_2^*, \ldots, \sigma_k^*\}$, where the partial set σ_i^* contains the element a_i and $m-1$ "$*$"s;
5'. change = TRUE;
6. *while* (change = TRUE) & (any set in P_k still contains "$*$" symbols) *do*
6'. change = FALSE;
6.1. call Algorithm II' on P_k;
6.1'. *if* Algorithm II' returns a set packing M and the partial set packing Q_k *then*
6.2. *if* $|M| = k$ *then* return (M); STOP;
6.4. pick a partial set σ^* in $Q_k - M$;
6.5. *if* there is a new element a in M such that replacing a "$*$" in σ^* by a gives a partial set consistent with at least one set in C
6.7. *then* replace a "$*$" by a in σ^* in the collection P_k; change = TRUE;
7. return("no such a packing"); STOP.

Fig. 5. The deterministic algorithm for set packing.

We analyze Algorithm IV. First consider the *while* loop starting from step 6 of Algorithm IV. We start with a partial set packing P_k with $k(m-1)$ "$*$"s (since each partial set in P_k has $m-1$ "$*$"s in step 5), and end with a partial set packing P_k with at least k "$*$"s (since Algorithm II' keeps each partial set in P_k with at least one "$*$", and when each partial set in P_k has exactly one "$*$", by Lemma 3.3, Algorithm II' will return a set packing M of k sets and Algorithm IV will stop at step 6.2). Since each execution of the *while* loop starting from step 6 converts a "$*$" in P_k into a non-"$*$", we conclude that the *while* loop starting from step 6 of Algorithm IV is executed at most $k(m-2)+1$ times.

Now consider the number of new elements in step 6.5 of Algorithm IV. At the beginning of the ith execution of the *while* loop starting from step 6, since $i-1$ "$*$"s in the original P_k have been converted by the previous executions of the *while* loop, the partial set packing P_k has $k(m-1)-i+1$ "$*$"s. Thus, the number of new elements in M at step 6.5 is bounded by $k(m-1)-i+1$. For each such a new element, step 6.7 of the algorithm may use it to replace a "$*$" and go to next execution of the *while* loop starting from step 6. Therefore, the total number of times that steps 6.5–6.7 are executed (for each spanning set for M_0 selected in step 4) is bounded by

$$\prod_{i=1}^{k(m-2)+1} \big[k(m-1)-i+1\big] = \frac{[k(m-1)]!}{(k-1)!} = \frac{k[k(m-1)]!}{k!}. \tag{4}$$

Using Stirling's approximation [9]: $n! \approx \sqrt{2n\pi}(n/e)^n$, we derive that the value in (4) is bounded by $k(m-1)(k^{m-2}(m-1)^{m-1}/e^{m-2})^k$.

The running time of the other parts in steps 6'–6.7 (i.e., steps 6'–6.4) is bounded by $O(k^3 + kmn)$ (here we have used Lemma 3.4). Since the *while* loop starting from step 6

is executed at most $k(m-2)+1$ times, for each spanning set $\{a_1, a_2, \ldots, a_k\}$ selected at step 4 of the algorithm, the total execution time of steps 6–6.7 is bounded by

$$O\left(k(m-1)\left(\frac{k^{m-2}(m-1)^{m-1}}{e^{m-2}}\right)^k + \big[k(m-2)+1\big]\big(k^3+kmn\big)\right).$$

By Lemma 3.2, there are at most b_m^k spanning sets of k elements for the k-maximal set packing M_0, where b_m is the minimum positive root of the equation in (3). Thus, the *for* loop starting from step 4 of Algorithm IV is executed at most b_m^k times. This, together with Lemma 2.1, concludes that the running time of the algorithm Algorithm IV is bounded by

$$O\left(b_m^k\left[k(m-1)\left(\frac{k^{m-2}(m-1)^{m-1}}{e^{m-2}}\right)^k + \big[k(m-2)+1\big]\big(k^3+kmn\big)\right] + mn\right)$$
$$= g(k,m)n$$

where $g(k,m)$ depends only on k and m.

Following the terminology in [7], a parameterized problem is *fixed parameter tractable* if it can be solved in time $O(f(k)n^c)$, where $f(k)$ is a function only depending on the parameter k, and c is a constant independent of both k and n. From our discussion above, we derive directly that for every fixed constant m, the m-PSP problem is fixed parameter tractable. In particular, for $m=3$, we can estimate the value $g(k,3)$ and obtain $g(k,3)n = O((5.7k)^k n)$ (here we have used the fact $b_3 < 3.8474$).

Theorem 3.5. *The m-PSP problem can be solved in time $g(k,m)n$, where $g(k,m)$ is a function depending only on k and m. For any fixed constant m, the m-PSP problem is fixed parameter tractable. In particular, the* 3*-PSP problem can be solved in time $O((5.7k)^k n)$.*

Acknowledgments

The authors thank Don Friesen, Iyad Kanj, and Ge Xia for their stimulating discussion and encouraging comments. The authors also thank the anonymous referees for their helpful comments that enable the improvements of the paper.

References

[1] R. Balasubramanian, M. Fellow, V. Raman, An improved fixed-parameter algorithm for vertex cover, Inform. Process. Lett. 65 (1998) 163–168.
[2] A. Bondy, U. Mutry, Graph Theory with Applications, North-Holland, New York, 1976.
[3] L. Cai, J. Chen, On fixed-parameter tractability and approximation of NP optimization problems, J. Comput. System Sci. 54 (1997) 465–474.
[4] L. Cai, J. Chen, R. Downey, M. Fellows, On the structure of parameterized problems in NP, Inform. and Comput. 123 (1995) 38–49.
[5] J. Chen, D.K. Friesen, W. Jia, I.A. Kanj, Using nondeterminism to design deterministic algorithms, Lecture Notes in Comput. Sci. 2245 (2001) 120–131.
[6] J. Chen, I. Kanj, W. Jia, Vertex cover: further observations and further improvements, J. Algorithms 41 (2001) 280–301.

[7] R. Downey, M. Fellows, Parameterized Complexity, Springer-Verlag, New York, 1999.
[8] M. Garey, D. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, Freeman, San Francisco, 1979.
[9] R. Graham, D. Knuth, O. Patashnik, Concrete Mathematics, Addison–Wesley, Reading, MA, 1994.
[10] J. Hopcroft, R. Karp, An $n^{5/2}$ algorithm for maximum matching in bipartite graphs, SIAM J. Comput. 2 (1973) 225–231.
[11] R. Karp, Reducibility among combinatorial problems, in: R. Miller, J. Thatcher (Eds.), Complexity of Computer Computations, Plenum Press, New York, 1972, pp. 85–103.
[12] C. Zhang, Numerical Methods, Educational and Scientific and Cultural Press, Hong Kong and China, 2001, ISBN 962-8467-47-6.